EXPERIMENT 6

Aim: Creation and Deployment of Smart Contract and Transactions using Solidity and Remix IDE.

Theory :

A smart contract is a self-executing program stored on a blockchain that automatically executes predefined instructions when specific conditions are satisfied. Smart contracts eliminate the need for intermediaries by enforcing rules through code. They are widely used in decentralized applications (DApps), cryptocurrency transactions, supply chain management, digital identity, and decentralized finance.

Solidity is a high-level, statically typed programming language used to develop smart contracts, primarily for Ethereum and other Ethereum Virtual Machine (EVM)-compatible blockchains. It supports features such as variables, functions, events, modifiers, and data structures to implement blockchain-based applications.

Remix IDE is a browser-based integrated development environment used to write, compile, deploy, and test Solidity smart contracts. It provides tools such as a Solidity compiler, a deployment interface, and a JavaScript VM or local blockchain environment for executing contracts and observing transactions without requiring a separate development setup.

Creation of a Smart Contract

The first step is to write the smart contract in Solidity using Remix IDE. A contract can contain state variables to store data, a constructor to initialize values, and functions to read or modify the stored data. Solidity supports access control through visibility specifiers such as `public`, `private`, `internal`, and `external`.

Compilation of the Smart Contract

After writing the contract, it is compiled using the Solidity compiler in Remix IDE. Compilation checks the source code for syntax and compilation errors and generates the contract's bytecode and Application Binary Interface (ABI). The bytecode is used for deployment on the blockchain, while the ABI defines how external applications interact with the contract.

Deployment of the Smart Contract

Deployment is the process of publishing the compiled smart contract to a blockchain. In Remix IDE, the contract can be deployed using the JavaScript VM for local testing or through a connected wallet to a supported testnet or network. During deployment, a transaction is generated, and the contract receives a unique blockchain address after successful deployment.

Execution of Smart Contract Transactions

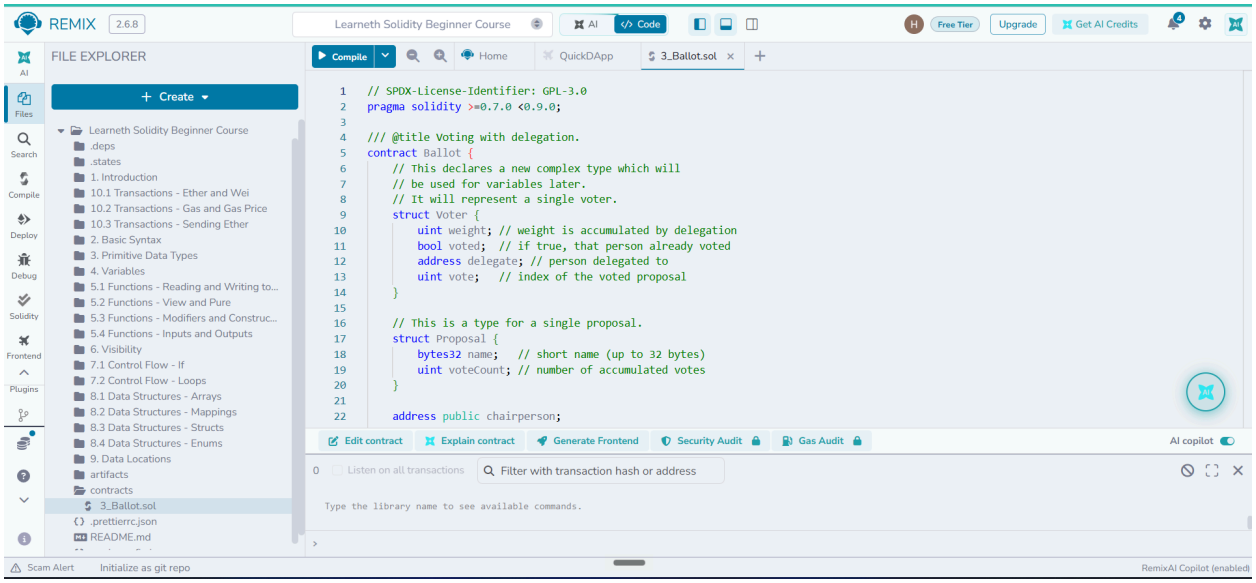
Once deployed, the contract's functions can be executed through Remix IDE. Read-only functions that do not modify blockchain state can generally be called without a transaction fee. Functions that modify the contract's state require a transaction, which consumes gas. Gas is the unit used to measure the computational work required to execute operations on an EVM-compatible blockchain.

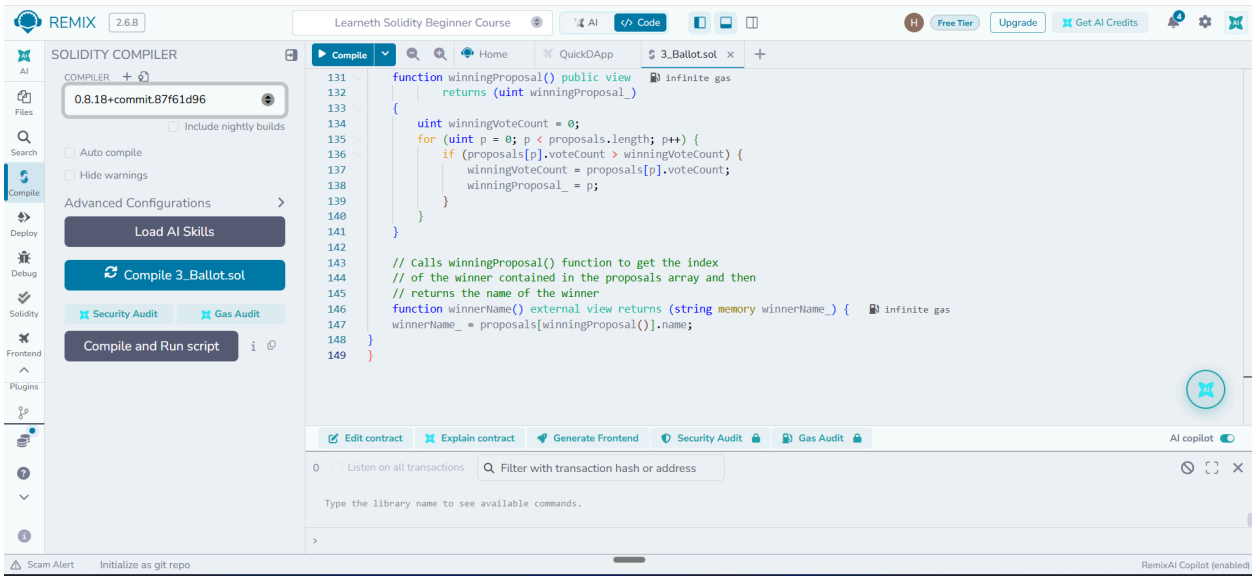
A transaction includes details such as the sender's address, destination address, input data, gas parameters, and, where applicable, the transferred value. The blockchain processes the transaction, executes the contract code, and records the resulting state changes. The transaction receipt provides information such as execution status and gas consumed.

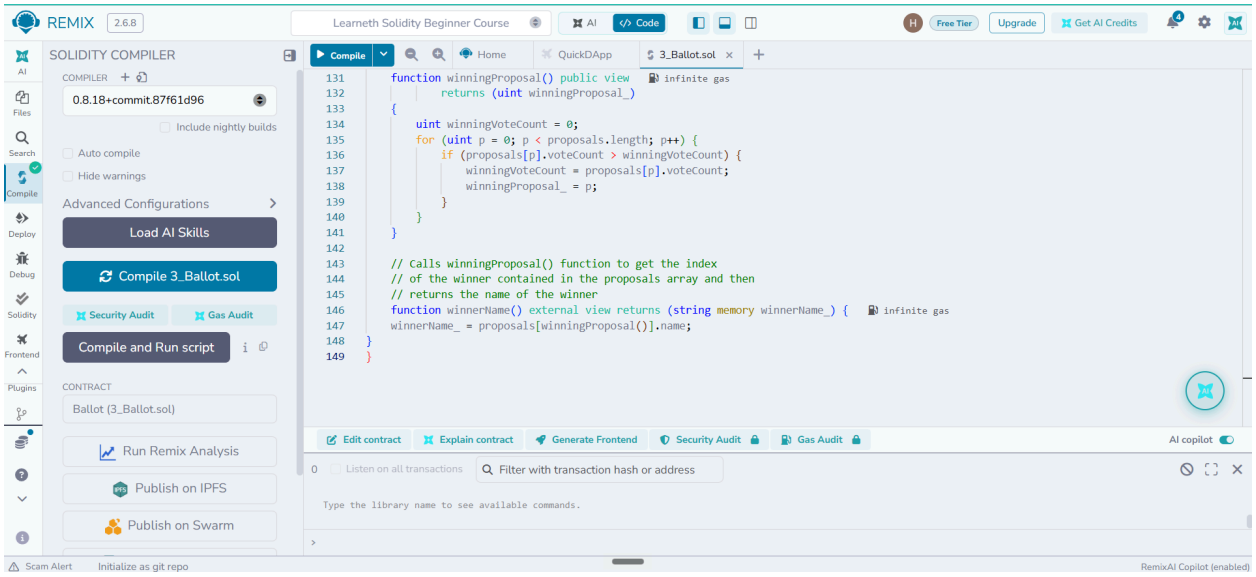
Role of Blockchain in Smart Contracts

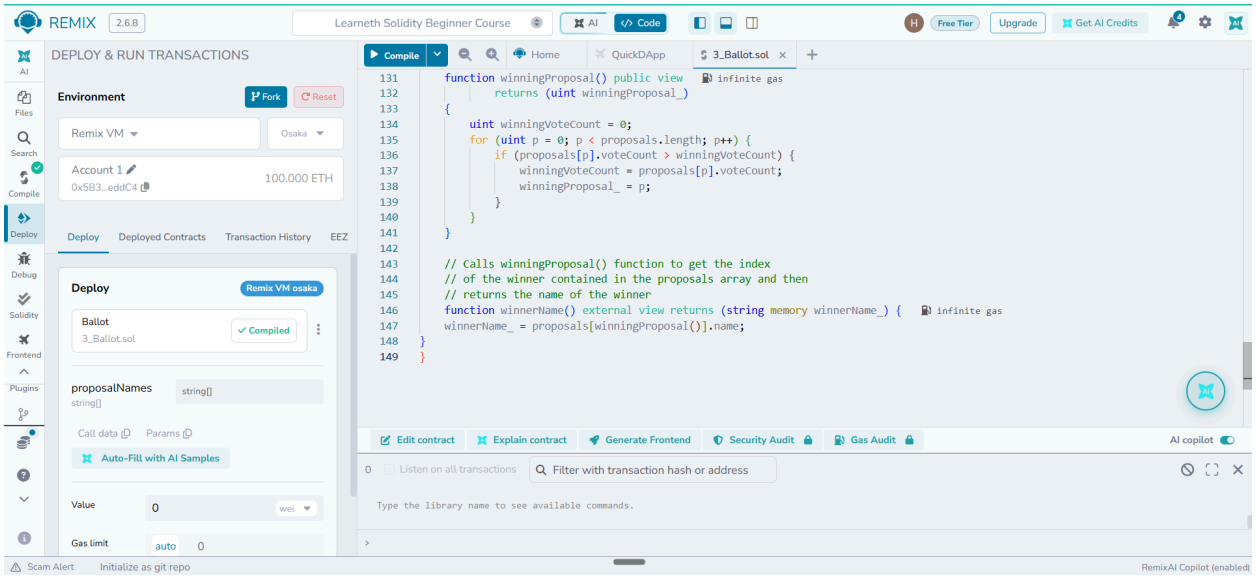
Blockchain technology provides decentralization, transparency, immutability, and security to smart contracts. Once a transaction is confirmed, its record is maintained on the blockchain and can be independently verified. However, smart contracts must be carefully designed and tested because coding errors may cause unintended behavior or financial losses.

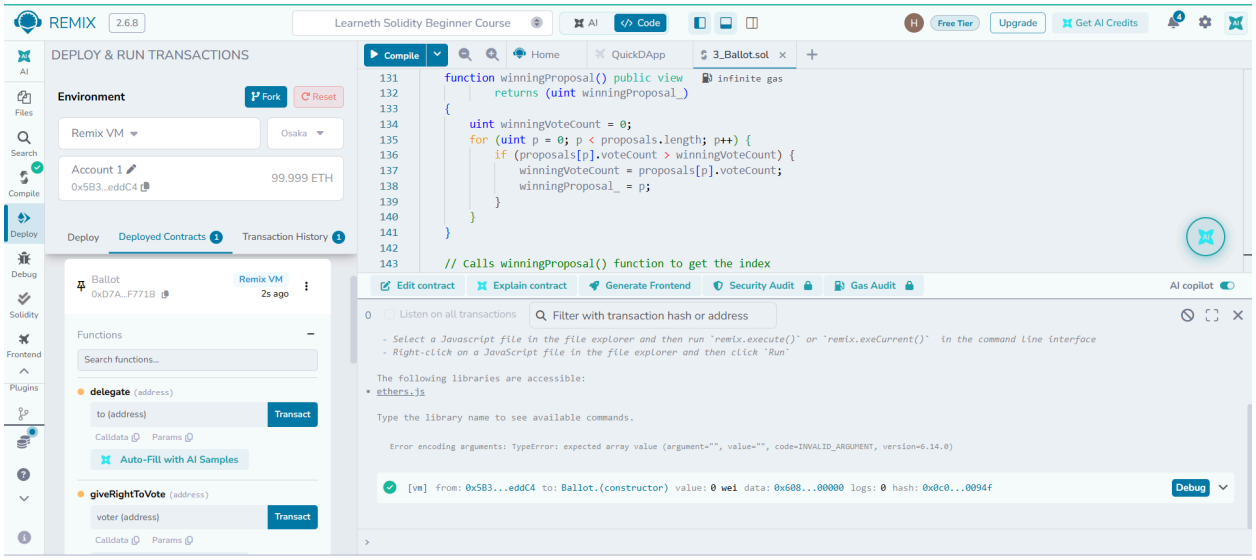
Code and Output :

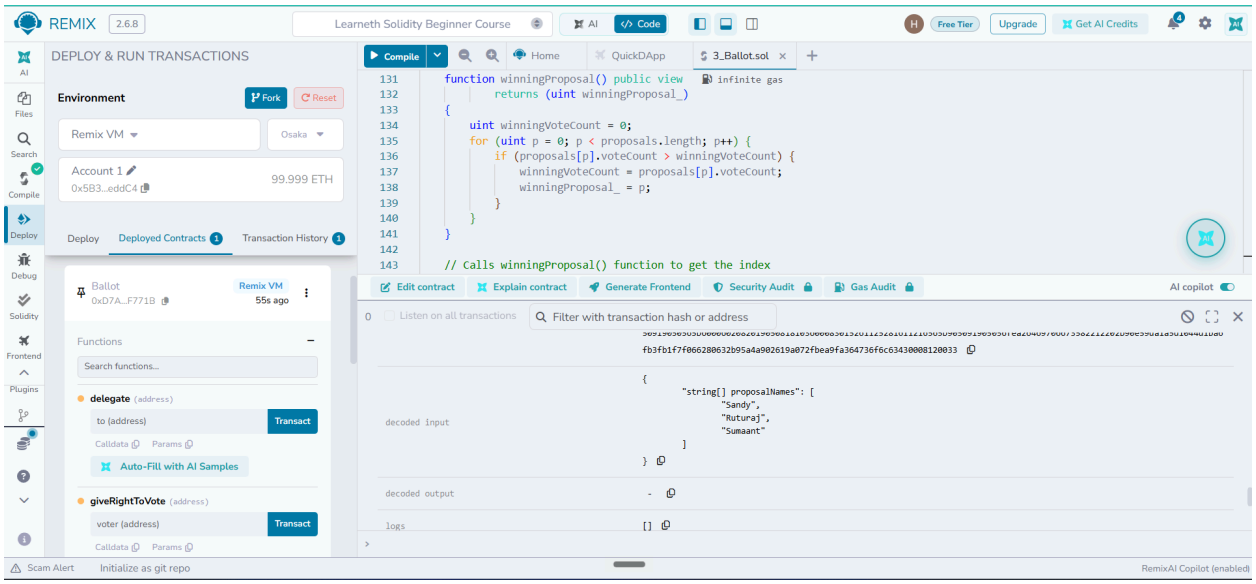





REMIX 2.6.8
Learneth Solidity Beginner Course
AI
Code
Free Tier
Upgrade
Get AI Credits
SOLIDITY COMPILER
COMPILER
0.8.18+commit.87f61d96
Include nightly builds
Auto compile
Hide warnings
Advanced Configurations
Load AI Skills
Compile 3_Ballot.sol
Security Audit
Gas Audit
Compile and Run script
CONTRACT
Ballot (3_Ballot.sol)
Run Remix Analysis
Publish on IPFS
Publish on Swarm
Compile
Home
QuickDAPP
3_Ballot.sol
function winningProposal() public view returns (uint winningProposal_)
{
uint winningVoteCount = 0;
for (uint p = 0; p < proposals.length; p++) {
if (proposals[p].voteCount > winningVoteCount) {
winningVoteCount = proposals[p].voteCount;
winningProposal_ = p;
}
}
}
// Calls winningProposal() function to get the index
// of the winner contained in the proposals array and then
// returns the name of the winner
function winnerName() external view returns (string memory winnerName_) {
winnerName_ = proposals[winningProposal()].name;
}
}
Edit contract
Explain contract
Generate Frontend
Security Audit
Gas Audit
AI copilot
Listen on all transactions
Filter with transaction hash or address
Type the library name to see available commands.
Scam Alert
Initialize as git repo
RemixAI Copilot (enabled)


REMIX 2.6.8
Learneth Solidity Beginner Course
AI
Code
Free Tier
Upgrade
Get AI Credits
DEPLOY & RUN TRANSACTIONS
Environment
Fork
Reset
Remix VM
Osaka
Account 1
0x5B3...eddC4
100.000 ETH
Deploy
Deployed Contracts
Transaction History
EEZ
Deploy
Remix VM osaka
Ballot
3_Ballot.sol
Compiled
proposalNames
string[]
Call data
Params
Auto-Fill with AI Samples
Value
0
wei
Gas limit
auto
0
Compile
Home
QuickDAPP
3_Ballot.sol
function winningProposal() public view returns (uint winningProposal_)
{
uint winningVoteCount = 0;
for (uint p = 0; p < proposals.length; p++) {
if (proposals[p].voteCount > winningVoteCount) {
winningVoteCount = proposals[p].voteCount;
winningProposal_ = p;
}
}
}
// Calls winningProposal() function to get the index
// of the winner contained in the proposals array and then
// returns the name of the winner
function winnerName() external view returns (string memory winnerName_) {
winnerName_ = proposals[winningProposal()].name;
}
}
Edit contract
Explain contract
Generate Frontend
Security Audit
Gas Audit
AI copilot
Listen on all transactions
Filter with transaction hash or address
Type the library name to see available commands.
Scam Alert
Initialize as git repo
RemixAI Copilot (enabled)





Conclusion :

The creation and deployment of smart contracts using Solidity and Remix IDE demonstrate how blockchain applications can automate transactions and securely maintain data. Remix IDE simplifies the development process by providing an integrated environment for writing, compiling, deploying, and testing smart contracts. This experiment helps in understanding Solidity programming, contract deployment, blockchain transactions, gas consumption, and the execution of decentralized applications.